

week4-day1-fine-tuning

.jsonl file upload:

Storage
Manage your uploaded files

1 TOTAL FILES | 1 READY FILES | 24.8 KB TOTAL STORAGE | Upload NEW FILE

FILE NAME	FILE ID	PURPOSE	SIZE	STATUS	CREATED
training_data_chat.jsonl	file-TyVPu1rstG4w52fy8kEn5	fine-tune	24.8 KB	Ready	23 May 2026 18:38

1-1 of 1 | 10 / page

Fine Tuning:

Fine-tuning
Train and manage custom models

1 TOTAL JOBS | 0 SUCCEEDED | 0 FAILED | 0 TRAINED TOKENS

All Successful Failed + Create

FINE-TUNE ID	BASE MODEL	METHOD	STATUS	TRAINED TOKENS	CREATED
gpt-4o-mini-2024-07-18	gpt-4o-mini-2024-07...	supervised	Running	-	23 May 2026 18:49

1-1 of 1 | 10 / page

Base Model Creation:

Create OPENAI - Testleaf GenAI API Platform

Fine Tuning

MENU

Usage

Cost

API Keys

Fine-tuning

Storage

AGENTS

Test Case Generator

Playwright Converter

Settings

Profile

Logout

Fine-tuning

Train and manage custom models

ft:gpt-4o-mini-2024-07-18:testleaf-2::DigdaTb0

Status

Job ID

Training Method

Base Model

Output Model

Created At

Succeeded

ftjob-U2jwC7ZnNQVwHRyQqcxgxnI

supervised

gpt-4o-mini-2024-07-18

ft:gpt-4o-mini-2024-07-18:testleaf-2::DigdaTb0

23 May 2026 18:49:33

Trained Tokens

Epochs

Batch Size

LR Multiplier

Seed

21,316

4

1

0.12

1630044234

21,316

TRAINED TOKENS

+ Create

ED	NS	CREATED
16		23 May 2026 18:49

1 of 1

< 1 >

10 / page

Create OPENAI - Testleaf GenAI API Platform

Fine Tuning

MENU

Usage

Cost

API Keys

Fine-tuning

Storage

AGENTS

Test Case Generator

Playwright Converter

Settings

Profile

Logout

Fine-tuning

Train and manage custom models

ft:gpt-4o-mini-2024-07-18:testleaf-2::DiivXxRw

Status

Job ID

Training Method

Base Model

Output Model

Created At

Succeeded

ftjob-7Vj9Ap8HJ6iHYeaPej2Pii3uU

supervised

gpt-4o-mini-2024-07-18

ft:gpt-4o-mini-2024-07-18:testleaf-2::DiivXxRw

23 May 2026 21:17:15

Trained Tokens

Epochs

Batch Size

LR Multiplier

Seed

21,316

4

1

0.6

668338467

42,632

TRAINED TOKENS

+ Create

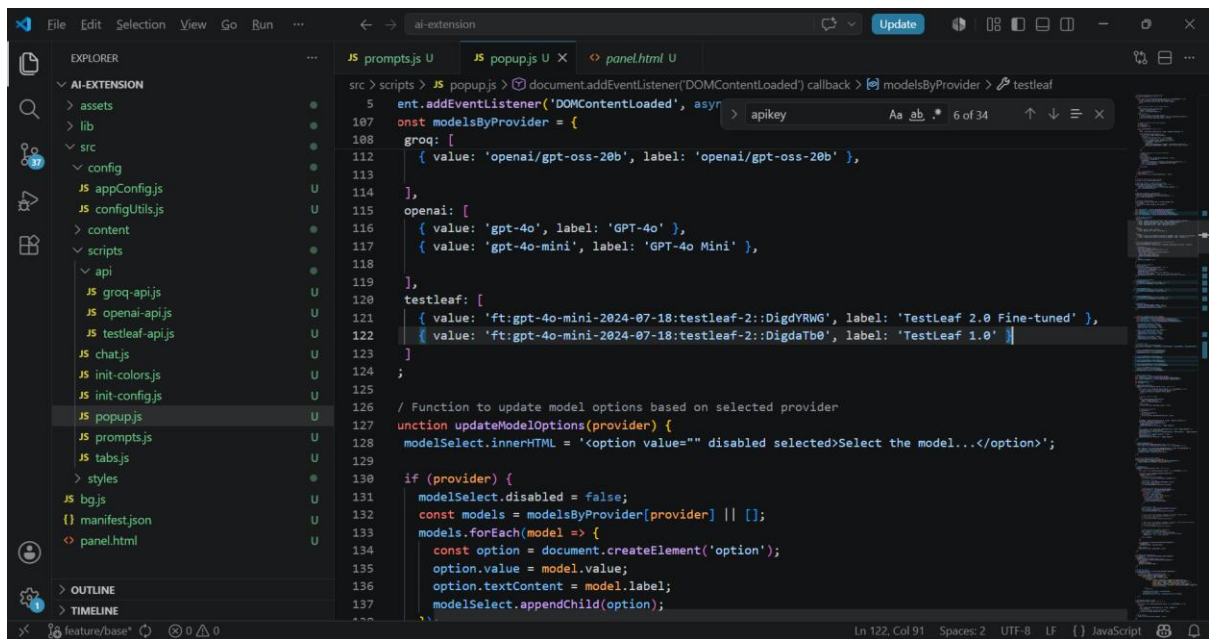
ED	NS	CREATED
16		23 May 2026 21:17
16		23 May 2026 18:49

2 of 2

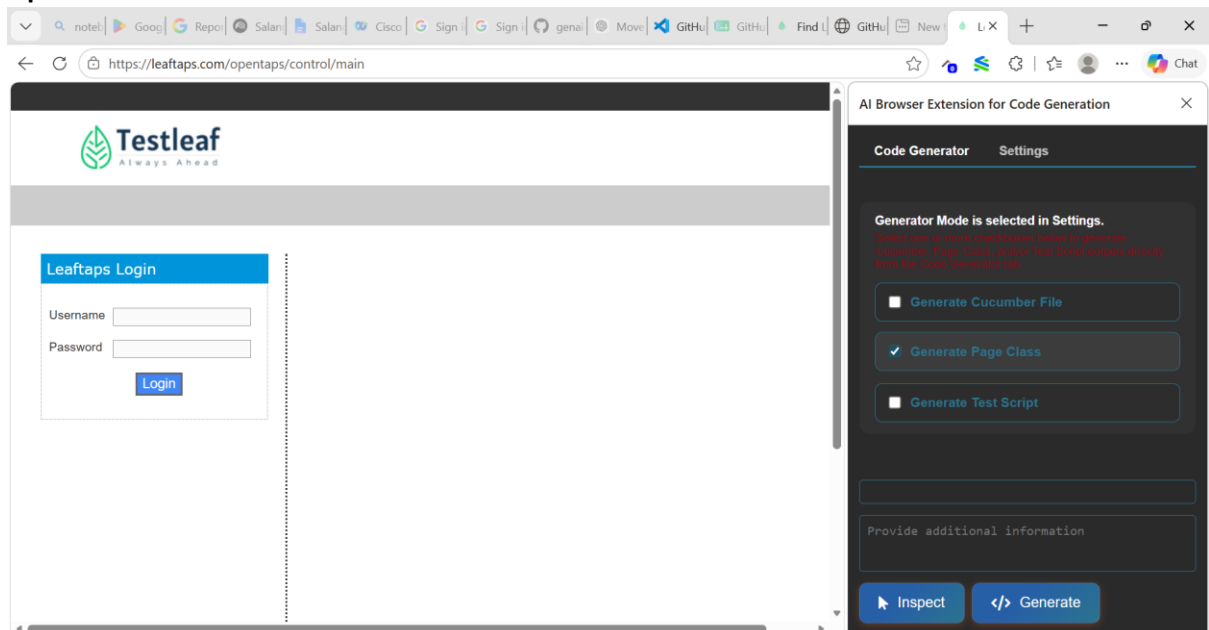
< 1 >

10 / page

Change popup.js on VSCode:



Open Browser Extension:



Change to Fine Tuned Model name on Settings:

The screenshot shows the LeafTaps control interface at <https://leafTaps.com/opentaps/control/logout>. On the left is the 'LeafTaps Login' form with fields for 'Username' and 'Password', and a 'Login' button. On the right is the 'AI Browser Extension for Code Generation' settings panel. The 'Settings' tab is active, showing the following configuration:

- Supported Language:** Java
- Browser Engine:** Selenium
- Generator Mode:** Selenium Java Page Only
- Select LLM Provider:** Testleaf
- Select Model:** TestLeaf 1.0
- Testleaf API Key:** [Redacted]

Inspect and Generate the Page Class:

The screenshot shows the LeafTaps control interface at <https://leafTaps.com/opentaps/control/main>. The 'LeafTaps Login' form is on the left. The 'AI Browser Extension for Code Generation' settings panel is on the right, with the 'Generator Mode' section expanded. It displays the message: 'Generator Mode is selected in Settings. Select how to interact with the browser. Generate Cucumber, Page Class, or Full Test Script output directly from the Code Generator tab.' Below this message are three buttons: 'Generate Cucumber File', 'Generate Page Class' (which is checked), and 'Generate Test Script'. At the bottom of the panel are 'Stop' and 'Generate' buttons.

Output Page Class:

The screenshot shows the LeafTaps web application with the AI Browser Extension for Code Generation open. The extension has three tabs: "Generate Cucumber File", "Generate Page Class" (selected), and "Generate Test Script". The "Reset Chat" button is visible. The code generation output is as follows:

```
package com.testleaf.pages;

import org.openqa.selenium.By;
import org.openqa.selenium.remote.RemoteWebDriver;

/**
 * Page Object representing the Login Page of LeafTaps application.
 * Provides methods to interact with the login form and perform login actions.
 */
public class LoginPage extends ProjectSpecificMethods {

    /**
     * Initializes the LoginPage with the given driver.
     */
}
```

Below the code, there is a text input field labeled "Provide additional information".

The screenshot shows the LeafTaps web application with the AI Browser Extension for Code Generation open. The extension has three tabs: "Generate Cucumber File", "Generate Page Class" (selected), and "Generate Test Script". The "Reset Chat" button is visible. The code generation output is as follows:

```
/**
 * Initializes the LoginPage with the given driver.
 */
* @param driver the RemoteWebDriver instance
*/
public LoginPage(RemoteWebDriver driver) {
    this.driver = driver;
}

/**
 * Enters the username into the login form.
 *
 * @param username the username to enter
 * @return the current LoginPage instance
 */
public LoginPage enterUsername(String username) {
    clearAndType(locatorElement("id", "username"), username);
}
```

Below the code, there is a text input field labeled "Provide additional information".

Testleaf
Always Ahead

LeafTaps Login

Username

Password

Login

AI Browser Extension for Code Generation

☐ Generate Cucumber File ☒ Generate Page Class ☐ Generate Test Script

Reset Chat

```
public LoginPage enterUsername(String username) {
    clearAndType(locatorElement("id", "username"), username);
    reportStep(username + " username is entered", "pass");
    return this;
}

/**
 * Enters the password into the login form.
 *
 * @param password the password to enter
 * @return the current LoginPage instance
 */
public LoginPage enterPassword(String password) {
    clearAndType(locatorElement("id", "password"), password);
    reportStep(password + " password is entered", "pass");
    return this;
}
```

Provide additional information

Testleaf
Always Ahead

LeafTaps Login

Username

Password

Login

AI Browser Extension for Code Generation

☐ Generate Cucumber File ☒ Generate Page Class ☐ Generate Test Script

Reset Chat

```
reportStep(password + " password is entered", "pass");
return this;
}

/**
 * Clicks the login button to submit the login form.
 *
 * @return the HomePage instance after login
 */
public HomePage clickLoginButton() {
    click(locatorElement("class", "decorativeSubmit"));
    reportStep("Login button is clicked", "pass");
    return new HomePage(driver);
}
}
```

Provide additional information